

万字详解 RAG 基础概念

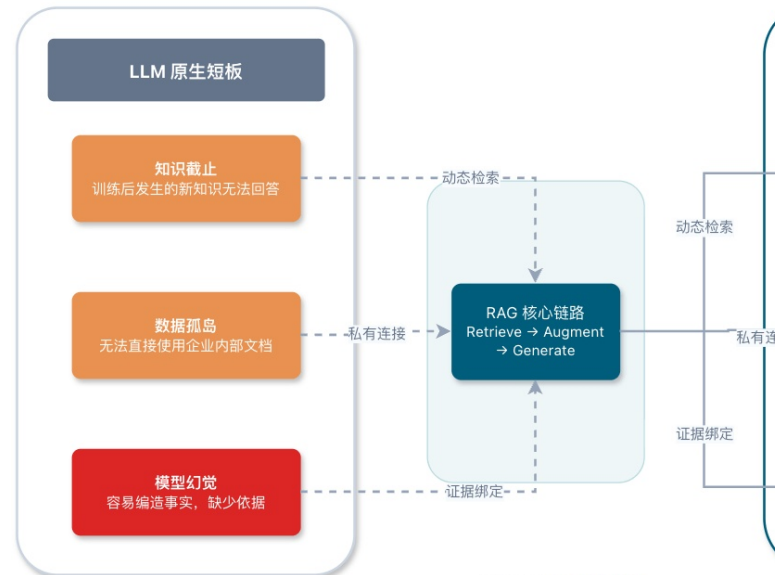
什么是 RAG ?

RAG (Retrieval-Augmented Generation, 检索增强生成)就是把信息检索和大语言模型绑在一起用。系统先从知识库里检索出和当前问题相关的片段，知识库可以是数据库、文档集合，也可以是企业内部系统。然后把这些片段和原始问题一起喂给 LLM，让模型基于检索内容回答，而不是只靠训练时记住的知识。

为什么需要 RAG ?

RAG 如何解决 LLM 的核心挑战

把“模型记忆”变成“外部知识检索 + 上下文增强 + 可追溯性”



公众号: [JavaGuide](#)
AI图解网站: [javaguide.cn](#)

结论: RAG 不是让模型“知道更多”, 而是让模型“先查证据, 再回答”

什么是 RAG ?

RAG (检索增强生成) 如何解决 LLM 的核心挑战

LLM 训练数据再大, 也绕不开几个问题。RAG 正好可以在这些地方进行弥补。

第一是知识时效性。

预训练模型的知识会停在训练数据截止时间点。训练后发生的新事件、新政策、新产品文档, 模型默认是不知道的, 除非通过联网、工具调用或外部知识注入来补。RAG 的做法是动态检索外部知识源, 把最新的相关内容直接送给 LLM, 让它不用只依赖参数里的知识。

第二是私有数据访问。

企业内部的文档、知识库、客户数据, 不可能让公开 LLM 随便访问。RAG 在用户提问时只提取和问题相关的片段给 LLM, 不需要暴露全部数据, 模型也能基于企业自己的知识回答。

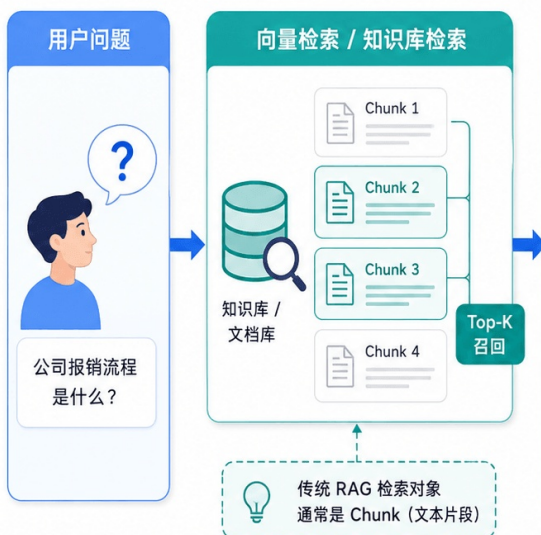
第三是幻觉问题。

LLM 会编造事实。RAG 通过提供明确参考文本, 让模型尽量基于证据回答, 能降低幻觉概率, 但不能彻底消除幻觉。检索错误、上下文噪声、引用错配、模型不遵循指令, 都可能导致错误答案。生产级 RAG 通常还要配引用校验、答案评估、拒答机制和人工反馈闭环。

RAG 的常见用途有哪些?

RAG 最适合“答案通常来自外部资料, 并且资料会变化或很长”的场景。它先从知识库检索相关内容, 再让大模型基于检索结果生成回答, 减少胡编, 同时提高可追溯性。常见场景包括这些:

- 客服机器人: 基于产品知识库做问答、排障、流程引导, 比如“如何退换货”“某型号设备报错码怎么处理”。



RAG 示意图

- 研发 / 运维 Copilot：检索代码库、接口文档、告警手册，辅助定位问题和生成修复建议。
- 医疗助手：检索指南、药品说明、院内规范后生成辅助建议，但不做最终诊断，比如“某药禁忌是什么”“依据指南解释检查指标含义”。
- 法律咨询：基于法规条文、案例、合同模板检索，生成条款解释和风险提示。
- 教育辅导：从教材、讲义、题库中检索知识点，生成讲解和例题步骤。
- 企业内部助手：连接制度、SOP、会议纪要、技术文档，做检索、总结、对比。
- 投研、合规、审计、销售方案支持：处理报告、披露、内控、产品手册、标书模板等资料。

为什么有些企业还是宁愿用传统搜索而不是 RAG？

不是所有问题都值得上 RAG。很多企业保留传统搜索，不是因为不知道 RAG 好用，而是用户需求本来就没到“生成答案”这一步。

如果用户只是想找一份制度原文、某个接口文档、一个合同模板，搜索框反而更直接。输入关键词，返回文档列表，用户自己点开确认，链路短、成本低、结果也更可控。RAG 则要先检索，再组织上下文，最后交给 LLM 生成答案。只要经过生成，就会多出延迟、Token 成本和总结偏差的风险。

所以选传统搜索还是 RAG，先看用户到底想要什么：是“帮我找到材料”，还是“帮我读完材料并给出结论”。

维度	传统搜索（搜索框）	RAG（检索 + 生成）
用户目标	找到文档、页面、附件	直接得到可读答案、总结或对比结论
延迟与成本	极低，容易扩展	更高，需要检索和 LLM 推理
可控性 / 可审计	强，直接给原文链接	弱一些，可能误解或总结偏差，需要引用与评测
风险	低，主要是召回排序问题	更高，包括幻觉、引用错误、越权泄露
数据治理	相对成熟，ACL、字段过滤都好做	更复杂，需要检索过滤、上下文脱敏、日志治理
适用场景	编号、标题、关键词检索，找模板、找制度原文	客服解答、技术排障、制度解读、跨文档总结对比
最佳实践	ES / BM25 + 权限过滤	混合检索 + 重排 + 引用溯源 + 权限过滤 + 评测闭环

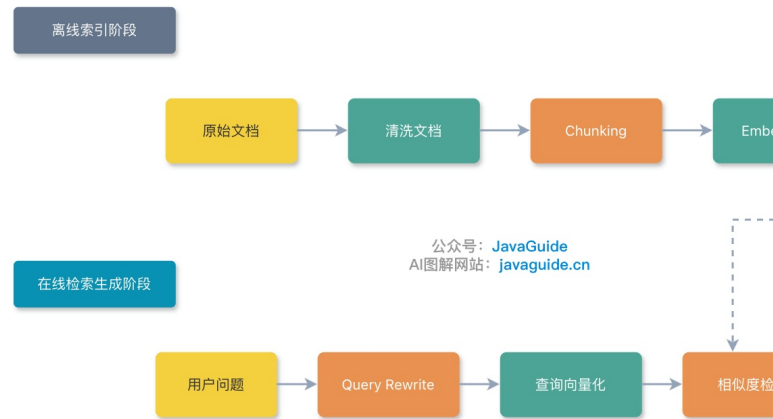
实际落地时，很多企业会同时保留两套入口：**简单查找走搜索，复杂问答走 RAG**。这个组合通常比“所有问题都交给 RAG”更稳，也更省钱。

RAG 工作原理了解吗？

RAG 的工程链路通常分两个阶段：离线索引和在线检索生成。索引阶段把原始文档处理成可检索的数据结构；在线阶段在用户提问时完成查询理解、检索召回、上下文构建和答案生成。

索引和检索阶段的简化流程图如下：

RAG 工程链路：离线建索引，在线检索生成
RAG 不是一个单点 API，而是一条完整工程链路：上游负责把文档变成可检索的向量，下游负责根据用户问题检索并生成答案。



索引和检索阶段的简化流程图

索引阶段主要做这些事：

1. 输入文档：文本文件、PDF、网页、数据库记录都可以，只要有内容。
2. 清理文档：去掉 HTML 标签、特殊字符等噪声。
3. 增强文档：补充元数据，比如时间戳、分类标签，为后续检索提供过滤维度。
4. 文档拆分 (Chunking)：用文本分割器把文档切成较小片段。这一步要兼顾语义完整性、Embedding 模型输入长度、生成模型上下文窗口和召回粒度。Chunk 太大容易引入噪声，太小又可能丢上下文。拆分策略会直接影响召回质量，详细可以看 RAG 文档处理篇。
5. 向量化表示 (Embedding Generation)：通过嵌入模型将文本片段映射为语义向量，也就是高维稠密向量。常见嵌入模型包括 OpenAI 的 `text-embedding-3-small` / `text-embedding-3-large`，以及 Hugging Face 上的开源模型。
6. 存储到向量存储或索引系统：把嵌入向量、原始内容和对应元数据存入向量存储或向量索引系统，比如 Milvus、pgvector、Elasticsearch / OpenSearch 向量检索，或基于 Faiss 构建本地向量索引。向量数据库选型、索引算法和 pgvector 实践可以看 RAG 向量库篇。

索引过程通常离线完成。比如团队每周跑一次定时任务，把新增和变更的文档重新索引一遍。如果是用户上传文档这类动态场景，索引也可以在线完成，直接集成到主应用里。检索是在线进行的。用户提问之后，系统通常会走下面这些步骤：

1. 接收请求：拿到用户的自然语言查询。有些系统会先做查询改写或扩充，让后续检索更容易命中。
2. 查询向量化：用嵌入模型把查询也转成向量，这样才能和文档向量在同一个空间里比较。
3. 信息检索 (R)：在向量库里做相似性搜索，把和查询向量最相关的文档片段捞出来。
4. 上下文增强 (A)：把检索片段、原始问题、系统指令和引用要求组织成 Prompt，交给 LLM。
5. 输出生成 (G)：LLM 输出自然语言回复，同时附上参考资料链接。

6. 结果反馈（可选）：用户不满意时可以反馈，系统再调整 Prompt 或检索策略。有些实现也支持多轮对话来逐步完善回答。

检索效果不稳定时，问题往往出在查询改写、召回策略、排序或上下文质量上。优化方向可以看 RAG 优化篇。

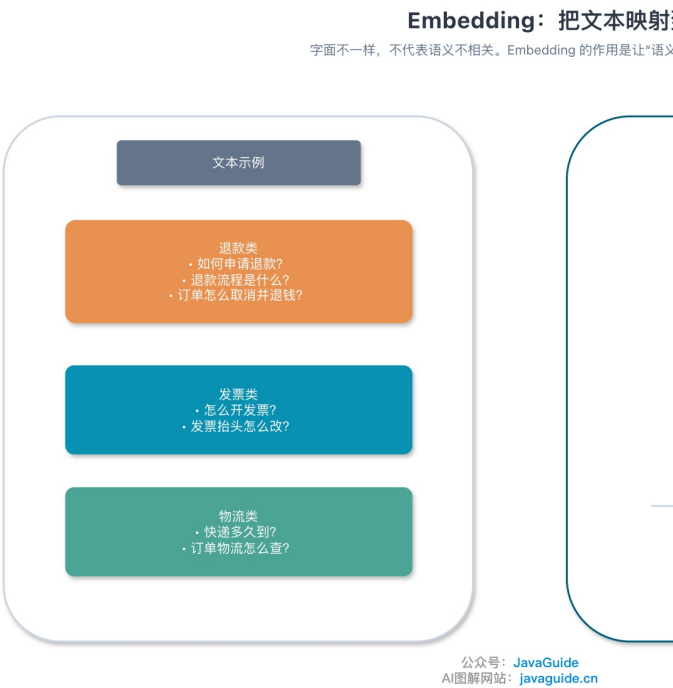
Embedding 是什么？

Embedding 就是把文本变成一串数字。更准确地说，它会把文本映射到一个高维稠密向量空间里，让语义接近的文本在向量空间中距离更近。

比如这三句话：

- “如何申请退款？”
- “退款流程是什么？”
- “订单怎么取消并退钱？”

它们字面不一样，但语义接近。好的 Embedding 模型会把它们映射到相近位置，向量检索才能把相关 Chunk 找出来。



Embedding 模型也不是“实时理解世界”的东西。它主要负责把文本映射到向量空间，能力重点是语义匹配。如果遇到非常新的术语、梗、产品名或领域缩写，仍然要通过业务语料评测确认召回效果。

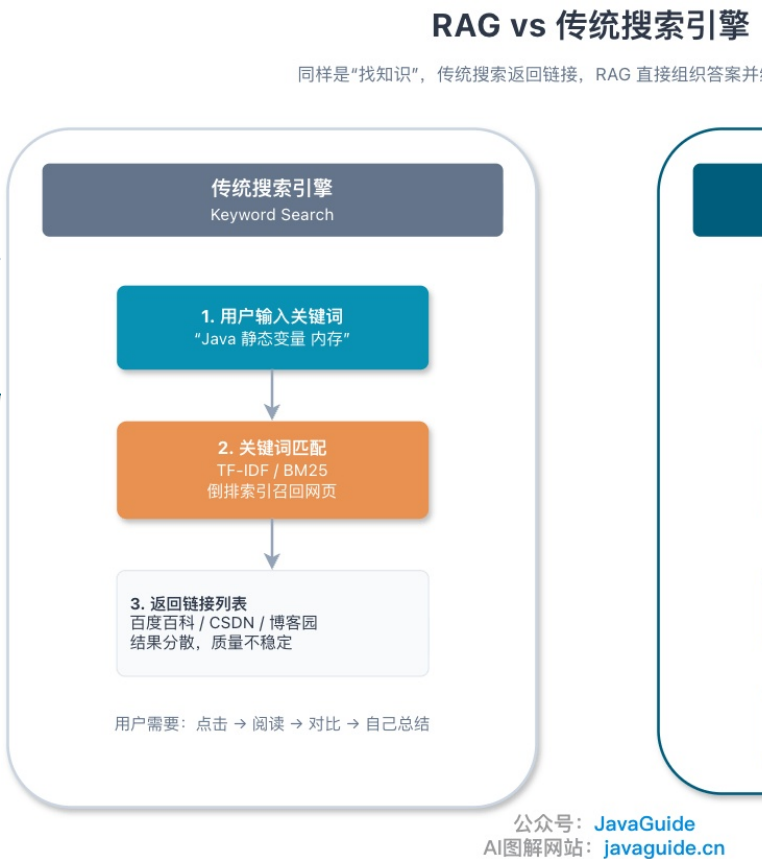
向量相似度怎么计算？

文本变成向量之后，检索系统还要判断哪个向量和查询最接近。常见相似度或距离度量有三种。

度量方式	含义	特点
余弦相似度 (Cosine Similarity)	看两个向量方向是否一致	对向量长度不敏感，RAG 场景最常用
内积 (Inner Product / Dot Product)	看两个向量对应维度乘积之和	如果向量已经 L2 归一化，内积和余弦相似度在排序结果上通常等价
欧氏距离 (L2 Distance)	看两个点在空间中的绝对距离	对向量幅度更敏感，适合模型或索引明确按 L2 训练 / 优化的场景

Embedding：把文本映射到语义空间。常用余弦相似度的原因：RAG 关注的是语义方向是否接近，而不是向量长度本身；余弦相似度对长度不敏感，更适合文本语义检索。实际项目里还要和 Embedding 模型推荐的距离度量、向量库索引类型保持一致，否则可能导致索引无法命中或召回效果下降。

RAG 与传统搜索引擎的区别是什么？



Embedding：把文本映射到语义空间

Embedding 维度通常是 768、1024、1536、3072 等。维度越高，能表达的信息越丰富，但存储、索引和相似度计算成本也越高。以 OpenAI Embedding 为例，text-embedding-3-small 默认输出 1536 维，text-embedding-3-large 默认输出 3072 维，并支持通过 dimensions 参数降低输出维度。常见 Embedding 模型可以分成两类：

类型	代表模型	适合场景
闭源	OpenAI text-embedding-3-small / text-embedding-3-large, Cohere Embed, API, Jina Embeddings API	追求开箱即用、多语言效果、少运维
开源模型	BGE 系列、GTE 系列、E5 系列、Jina Embeddings 开源模型	数据不能出内网、需要私有化部署、希望控制成本

选 Embedding 模型时，别只看榜单排名。MTEB (Massive Text Embedding Benchmark) 可以作为参考，但最后还是要用自己的业务问题评测召回率、相关性和延迟。

RAG 和传统搜索都在“找信息”，但拿到信息之后做的事不一样。传统搜索拿到候选文档后，按相关性排好序，直接把结果列表给用户。每个结果彼此独立，用户自己点开、自己判断。它更像一个排序器。RAG 会把检索到的多个知识片段一起放进 LLM 上下文，让模型做跨文档归纳和信息整合，最后生成一个直接能读的答案。它更像一个信息综合器。

几个差异比较关键：

1. 检索机制：传统搜索主要靠倒排索引和关键词匹配，BM25 是经典算法；现代搜索系统也会加语义召回和重排。RAG 的检索方式更灵活，向量检索、BM25、混合检索、图检索、数据库查询都可以用，关键是检索结果要进入 LLM 上下文参与答案生成。
2. 结果形态：搜索给文档列表，用户还要二次阅读；RAG 给答案，并尽量标出引用来源。
3. 数据范围：传统搜索擅长全网爬虫和大规模索引；RAG 更常用于企业内部知识库和垂直领域，让 LLM 低成本获得特定领域知识补充。
4. 成本和延迟：搜索响应快，成本可控；RAG 多了 LLM 推理，延迟和成本都会上去。

RAG 和微调怎么选？

RAG 与微调的选型问题很常见。二者区分：RAG 解决模型缺少新知识或私有知识的问题，微调解决输出风格、格式和任务行为不符合要求的问题。类比：员工手册经常需要查阅。RAG 是随查随用，把手册放在外部，每次回答前检索；微调是把手册内容内化进模型。手册频繁改版时，RAG 更新索引即可；微调需要重新准备数据、训练和评测，成本差异明显。

维度	RAG	微调 (Fine-tuning)
知识更新	更新知识库或向量索引即可	通常需要重新准备数据并训练
数据安全	知识保留在外部库，按需检索	训练样本中的模式和部分知识会固化到微调模型参数中，敏感数据进入训练流程前需要额外评估合规和数据治理要求
幻觉控制	可引用原文，便于溯源和校验	模型仍可能编造，且引用来源不天然可见
成本结构	检索成本 + 输入 Token 成本 + 向量库成本	数据标注、训练 GPU、评测和版本管理成本
适合场景	知识密集型问答、企业知识库、法规制度、产品文档、实时信息	风格适配、格式控制、领域术语对齐、固定任务行为优化
主要风险	检索不到、召回噪声、权限过滤复杂	数据过拟合、知识过期、训练和回滚成本高

二者也可以结合。先用微调让模型更懂领域术语、输出格式和任务边界，再用 RAG 提供实时知识和可追溯证据。这类组合在客服、法律、医疗、金融投研等场景里很常见。选型结论：知识变动频繁、需要引用来源，优先 RAG；输出风格和任务行为不稳定，考虑微调；既要懂领域表达又要查实时知识，可以两者结合。不过这里有个现实限制：两者结合意味着两套系统都要维护，成本不低。团队资源有限时，先把 RAG 做稳，再考虑是否引入微调，通常更务实。

长上下文窗口会取代 RAG 吗？

不会。长上下文窗口确实让很多任务变简单了。比如把一整份报告丢进去，让模型从头读到尾，这类单文档深度分析很适合用长上下文。但它不等于可以把全部知识库都塞给模型。上下文越长，输入 Token 成本、首字延迟和推理噪声都会上升，效果未必更好。长上下文适合的场景很明确：单篇长文档深度分析，一个代码仓库或一个项目目录的集中理解，长对话历史总结，或者一次性材料不多但需要完整阅读的任务。知识库规模一大，长上下文就不够用了。企业知识库、客服工单、日志、合同库动辄百万到亿级文档片段，不可能每次

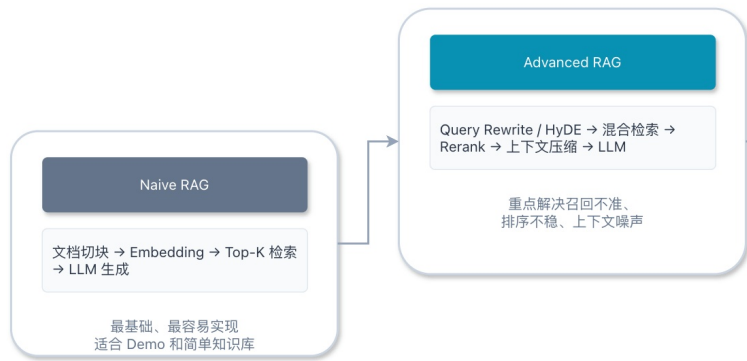
都全塞进去。就算塞得进去，成本和延迟也扛不住。更麻烦的是，上下文里塞太多无关片段，模型反而更容易被噪声干扰，生成看起来完整但事实不稳的答案。“Lost in the Middle”问题说的就是这个，关键信息放在长上下文中间位置时更容易被忽略。企业知识库还绕不开权限隔离。哪些内容用户能看，哪些不能看，不能靠“全塞进去”解决。RAG 可以在检索阶段做权限过滤，只把用户有权访问的内容放进上下文。长上下文做不了这件事。还有一点经常被忽视：可追溯性。RAG 可以明确返回引用片段，审计时能溯源。长上下文把大量内容混在一起交给模型，用户很难判断回答到底基于哪段材料。

RAG 有哪些演进阶段？

RAG 这两年一直在迭代，大致可以分成三个阶段。

RAG 的三个演进阶段

从能跑 Demo 的 Naive RAG，到强召回质量的 Advanced RAG，再到按场景定制的 Modular RAG



RAG 演进阶段

阶段	典型链路	特点
Naive RAG	文档切块 → Embedding → Top-K 检索 → LLM 生成	最基础、最容易实现，适合 Demo 和简单知识库
Advanced RAG	Query Rewrite / HyDE → 混合检索 → Rerank → 上下文压缩 → LLM 生成	重点解决召回不准、上下文噪声和排序不稳
Modular RAG	检索器、重排器、压缩器、路由器、生成器等模块可插拔组合	按业务场景动态路由，适合生产系统和复杂 Agent

Naive RAG 是起点，能跑通 Demo，但离生产通常还有距离。Advanced RAG 开始处理召回质量、噪声过滤和排序问题。Modular RAG 把各环节拆成可替换模块，更适合复杂场景。具体优化策略可以继续看 RAG 优化篇。

RAG 的核心优势和局限性是？

先说优势。**RAG 最大的好处是知识更新成本低。**微调要重新准备数据、训练模型、评测效果，RAG 通常只需要更新知识库和索引。新闻、法规、产品文档这类经常变化的数据，用 RAG 维护起来会轻很多。**它也能减少幻觉，并且方便追溯来源。**RAG 让模型从“凭记忆回答”变成“基于检索证据回答”。每个回答都可以挂到具体文档片段上，这在金融合规、医疗辅助、法律检索这些对准确性要求高的场景里很重要。当然，这不代表 RAG 就不会出错，检索错了、引用错了，答案一样会翻车。**数据隔离也更容易做。**检索层可实现多租户隔离和访问控制 (ACL)，确保用户只能看到权限范围内的数据。相比把

敏感数据放进微调训练集，RAG 这套架构更适合做权限和合规治理。

换领域的成本也低。不需要针对每个领域重新训练模型，把领域知识库建好、索引跑通，就能先用起来。

再看局限。RAG 不是银弹，坑也不少。

检索质量决定上限。GIGO 原则在这里特别明显：如果 Embedding 表达不准，或者分块策略把关键信息切丢了，召回内容和问题本身无关，下游 LLM 再强也救不回来。

上下文也不是越长越好。虽然有些模型的 Context Window 已经扩展到百万级，但塞太多无关片段进去，模型注意力会被稀释，逻辑推理会被干扰，Token 开销也会跟着上升。

延迟是另一个硬问题。完整链路要经过查询改写、向量化、相似度检索、重排序、上下文构建、LLM 生成，每一步都会增加耗时。对响应时间敏感的场景，不能只看答案质量，也要认真算延迟账。

工程复杂度也不低。需要维护向量数据库，处理文档增量索引，持续优化检索策略，并完成权限过滤、引用溯源和评测闭环。相比直接调用 LLM API，RAG 的运维负担明显更重。

Token 成本同样要算清楚。RAG 省了训练成本，但每次请求都要带上下文，输入 Token 往往比普通对话高不少。文档片段塞得越多，账单和延迟都会一起涨。

RAG 实战项目推荐

参考项目：Spring Boot 4.0 + Java 21 + Spring AI + PostgreSQL + pgvector + RustFS + Redis，覆盖简历解析、岗位匹配、知识库 RAG。

项目地址：<<https://github.com/Snailclimb/interview-guide>> · <<https://gitee.com/SnailClimb/interview-guide>>

总结

RAG 先从知识库检索相关内容，再让 LLM 基于检索结果回答。其价值不是提升模型能力，而是把回答拉回可检索、可引用、可审计的证据上。

关键点：

1. RAG 主要解决的是 LLM 知识过时、碰不到私有数据、容易幻觉这几个问题。传统搜索给的是文档列表，RAG 给的是直接可读的答案；一个更像排序器，一个更像信息综合器。
2. 知识变动频繁、需要引用来源时，优先考虑 RAG；如果要让模型按固定风格和格式输出，再考虑微调。
3. 长上下文适合少量材料的深度分析，但企业级海量知识库、权限隔离和成本控制，还是要靠 RAG 这类检索链路来兜底。

其局限同样明确：检索质量决定上限，上下文噪声会干扰生成，延迟、工程复杂度和 Token 成本真实存在。

Demo 跑通不等于生产可用；RAG 的难点通常不是接入向量库，而是持续评估和优化召回质量。